

# BSFM DATABASE PROJECT

## Staff, Assignment and Invoice

*Database design and SQL queries for a cleaning and facilities management booking system*

| Field             | Detail                                                                                                                          |
|-------------------|---------------------------------------------------------------------------------------------------------------------------------|
| Name              | S M Shakhawat Azam                                                                                                              |
| Student ID        | 0222220005101194                                                                                                                |
| Course            | Internship                                                                                                                      |
| Project website   | <a href="https://bsfm.ae/">https://bsfm.ae/</a>                                                                                 |
| GitHub repository | <a href="https://github.com/shakhawat1194/bsfm-shakhawat-database">https://github.com/shakhawat1194/bsfm-shakhawat-database</a> |

# 1. Introduction

BSFM is a cleaning and facilities management company (<https://bsfm.ae/>). This project continues the design and query work of the BSFM booking database, covering the part of the system that tracks staff, their assignments to bookings, and the invoices raised for completed work.

This is design and query work only. Nothing was implemented on the live website. The database was built and tested on a local computer, in the same bsfm\_db database used for the Customer, Service and Booking tables.

The task was to:

- Design and create the Staff, Assignment and Invoice tables.
- Store staff details, link staff to bookings through assignments, and record invoice amounts and payment status.
- Perform the four basic database operations: create, read, update and delete rows.
- Use JOIN queries to show a staff member's assignment history and a list of unpaid invoices with the customer's name.

The complete database has six tables. Customer, Service and Booking were designed and documented separately. This report covers Staff, Assignment and Invoice, which link into the same Booking table.

## 2. Tools and Environment

The same local environment used for the rest of the database was used here.

| Tool            | Purpose                                                                                          |
|-----------------|--------------------------------------------------------------------------------------------------|
| XAMPP           | A package that installs the Apache web server and the MySQL database server on a local computer. |
| phpMyAdmin      | A web page used to create the database and to run SQL commands.                                  |
| MySQL (MariaDB) | The database engine. The database is named bsfm_db and uses the utf8mb4 character set.           |
| GitHub          | Stores the SQL scripts and screenshots in an online repository.                                  |

## 3. Database Design

### 3.1 Overview of the Three Tables

| Table      | What it stores                                                                                                 |
|------------|----------------------------------------------------------------------------------------------------------------|
| Staff      | One row per staff member: name, phone number, role and hire date.                                              |
| Assignment | One row per job assignment. It links one staff member to one booking, with the date and their role on the job. |
| Invoice    | One row per invoice. It links to one booking and stores the amount, issue date and payment status.             |

### 3.2 How the Tables Connect

Staff and Booking are the two tables that Assignment links together: one staff member can be assigned to many bookings, and one booking can have more than one staff member assigned to it. Invoice links to Booking directly: each booking can have at most one invoice, enforced by a UNIQUE constraint on the booking\_id column of Invoice.

- One staff member can have many assignments. Each assignment belongs to exactly one staff member.

- One booking can have many assignments (more than one staff member on the same job) but at most one invoice.
- The foreign keys protect the data: an assignment cannot be created for a staff member or a booking that does not exist, and neither can an invoice.

### 3.3 Column Definitions

**STAFF TABLE**

| Column    | Data type    | Rules                       | Purpose                                                                             |
|-----------|--------------|-----------------------------|-------------------------------------------------------------------------------------|
| staff_id  | INT          | Primary key, numbers itself | Unique ID of each staff member                                                      |
| full_name | VARCHAR(100) | Required                    | Staff member's full name                                                            |
| phone     | VARCHAR(20)  | Required, must be unique    | Contact number. The unique rule stops the same staff member being registered twice. |
| role      | VARCHAR(50)  | Required                    | Job role, e.g. Cleaner, Technician, Supervisor                                      |
| hire_date | DATE         | Optional                    | Date the staff member was hired                                                     |

**ASSIGNMENT TABLE**

| Column        | Data type   | Rules                            | Purpose                                                   |
|---------------|-------------|----------------------------------|-----------------------------------------------------------|
| assignment_id | INT         | Primary key, numbers itself      | Unique ID of each assignment                              |
| booking_id    | INT         | Required, foreign key to Booking | Which booking the staff member is working on              |
| staff_id      | INT         | Required, foreign key to Staff   | Which staff member is assigned                            |
| assigned_date | DATE        | Required                         | Date of the assignment                                    |
| role_in_job   | VARCHAR(50) | Optional                         | The staff member's role on this job, e.g. Lead, Assistant |

**INVOICE TABLE**

| Column         | Data type     | Rules                                     | Purpose                                                                        |
|----------------|---------------|-------------------------------------------|--------------------------------------------------------------------------------|
| invoice_id     | INT           | Primary key, numbers itself               | Unique ID of each invoice                                                      |
| booking_id     | INT           | Required, foreign key to Booking, unique  | Which booking is being invoiced. Unique so a booking cannot be invoiced twice. |
| amount         | DECIMAL(10,2) | Required                                  | Invoice amount, with two decimal places                                        |
| issue_date     | DATE          | Required                                  | Date the invoice was issued                                                    |
| payment_status | ENUM          | Unpaid, Paid or Refunded. Default: Unpaid | Current payment state of the invoice. ENUM allows only these three values.     |

## 4. Table Structures in phpMyAdmin

The screenshots below show the three tables as they were created in phpMyAdmin.

### 4.1 Staff

The gold key marks staff\_id as the primary key. The grey key beside phone marks the unique rule. Only hire\_date allows empty values.

Server: 127.0.0.1 » Database: bsfm\_db » Table: staff

Table structure | Relation view

| # | Name      | Type         | Collation          | Attributes | Null | Default | Comments | Extra          | Action           |
|---|-----------|--------------|--------------------|------------|------|---------|----------|----------------|------------------|
| 1 | staff_id  | int(11)      |                    |            | No   | None    |          | AUTO_INCREMENT | Change Drop More |
| 2 | full_name | varchar(100) | utf8mb4_general_ci |            | No   | None    |          |                | Change Drop More |
| 3 | phone     | varchar(20)  | utf8mb4_general_ci |            | No   | None    |          |                | Change Drop More |
| 4 | role      | varchar(50)  | utf8mb4_general_ci |            | No   | None    |          |                | Change Drop More |
| 5 | hire_date | date         |                    |            | Yes  | NULL    |          |                | Change Drop More |

Indexes

| Action           | Keyname | Type  | Unique | Packed | Column   | Cardinality | Collation | Null | Comment |
|------------------|---------|-------|--------|--------|----------|-------------|-----------|------|---------|
| Edit Rename Drop | PRIMARY | BTREE | Yes    | No     | staff_id | 4           | A         | No   |         |
| Edit Rename Drop | phone   | BTREE | Yes    | No     | phone    | 4           | A         | No   |         |

Figure 1: Structure of the Staff table

## 4.2 Assignment

booking\_id and staff\_id are the foreign keys, and the grey key icons beside them mark the indexes MySQL creates for them.

Server: 127.0.0.1 » Database: bsfm\_db » Table: assignment

Table structure | Relation view

| # | Name          | Type        | Collation          | Attributes | Null | Default | Comments | Extra          | Action           |
|---|---------------|-------------|--------------------|------------|------|---------|----------|----------------|------------------|
| 1 | assignment_id | int(11)     |                    |            | No   | None    |          | AUTO_INCREMENT | Change Drop More |
| 2 | booking_id    | int(11)     |                    |            | No   | None    |          |                | Change Drop More |
| 3 | staff_id      | int(11)     |                    |            | No   | None    |          |                | Change Drop More |
| 4 | assigned_date | date        |                    |            | No   | None    |          |                | Change Drop More |
| 5 | role_in_job   | varchar(50) | utf8mb4_general_ci |            | Yes  | NULL    |          |                | Change Drop More |

Indexes

| Action           | Keyname    | Type  | Unique | Packed | Column        | Cardinality | Collation | Null | Comment |
|------------------|------------|-------|--------|--------|---------------|-------------|-----------|------|---------|
| Edit Rename Drop | PRIMARY    | BTREE | Yes    | No     | assignment_id | 4           | A         | No   |         |
| Edit Rename Drop | booking_id | BTREE | No     | No     | booking_id    | 4           | A         | No   |         |
| Edit Rename Drop | staff_id   | BTREE | No     | No     | staff_id      | 4           | A         | No   |         |

Figure 2: Structure of the Assignment table

## 4.3 Invoice

The grey key beside booking\_id marks the unique rule, so a booking cannot have more than one invoice. The payment\_status column defaults to Unpaid.

The screenshot displays the phpMyAdmin interface for the 'bsfm\_db' database, specifically the 'Table structure' view for the 'invoice' table. The table structure is as follows:

| # | Name           | Type                               | Collation          | Attributes | Null | Default | Comments | Extra          | Action           |
|---|----------------|------------------------------------|--------------------|------------|------|---------|----------|----------------|------------------|
| 1 | invoice_id     | int(11)                            |                    |            | No   | None    |          | AUTO_INCREMENT | Change Drop More |
| 2 | booking_id     | int(11)                            |                    |            | No   | None    |          |                | Change Drop More |
| 3 | amount         | decimal(10,2)                      |                    |            | No   | None    |          |                | Change Drop More |
| 4 | issue_date     | date                               |                    |            | No   | None    |          |                | Change Drop More |
| 5 | payment_status | enum('Unpaid', 'Paid', 'Refunded') | utf8mb4_general_ci |            | No   | Unpaid  |          |                | Change Drop More |

Below the table structure, the 'Indexes' section shows the following:

| Action           | Keyname | Type  | Unique | Packed | Column     | Cardinality | Collation | Null | Comment |
|------------------|---------|-------|--------|--------|------------|-------------|-----------|------|---------|
| Edit Rename Drop | PRIMARY | BTREE | Yes    | No     | invoice_id | 3           | A         | No   |         |
| Edit Rename Drop |         | BTREE | Yes    | No     | booking_id | 3           | A         | No   |         |

Figure 3: Structure of the Invoice table

## 5. Setup and Sample Data

### 5.1 Creating the Tables

The Staff, Assignment and Invoice tables were created in the same bsfm\_db database that already held Customer, Service and Booking (built separately). They were created in this order: Staff, then Assignment and Invoice, because both refer to Booking and Assignment also refers to Staff.

### 5.2 Sample Data

This is the sample data that was loaded before the queries were run.

#### STAFF

| ID | Full name    | Phone      | Role       | Hire date  |
|----|--------------|------------|------------|------------|
| 1  | Yusuf Rahman | 0509990001 | Cleaner    | 2025-01-15 |
| 2  | Sara Khaled  | 0509990002 | Technician | 2025-03-10 |
| 3  | Hassan Ali   | 0509990003 | Supervisor | 2024-11-01 |

#### ASSIGNMENT

| ID | Booking ID | Staff ID | Assigned date | Role in job |
|----|------------|----------|---------------|-------------|
| 1  | 1          | 1        | 2026-09-25    | Lead        |
| 2  | 2          | 2        | 2026-09-26    | Lead        |
| 3  | 3          | 1        | 2026-09-27    | Assistant   |
| 4  | 4          | 3        | 2026-09-28    | Lead        |

#### INVOICE

| ID | Booking ID | Amount | Issue date | Payment status |
|----|------------|--------|------------|----------------|
| 1  | 1          | 350.00 | 2026-09-25 | Paid           |
| 2  | 3          | 200.00 | 2026-09-27 | Paid           |
| 3  | 4          | 900.00 | 2026-09-28 | Unpaid         |

Figure 4 shows the row counts for all six tables in bsfm\_db after this sample data was loaded (before Query 1 was run): 3 customers, 4 services, 5 bookings, 3 staff, 4 assignments and 3 invoices, for 22 rows in total.

| Table                               | Action     | Rows      | Type          | Collation                 | Size             | Overhead   |
|-------------------------------------|------------|-----------|---------------|---------------------------|------------------|------------|
| <input type="checkbox"/> assignment |            | 4         | InnoDB        | utf8mb4_general_ci        | 48.0 KiB         | -          |
| <input type="checkbox"/> booking    |            | 5         | InnoDB        | utf8mb4_general_ci        | 48.0 KiB         | -          |
| <input type="checkbox"/> customer   |            | 3         | InnoDB        | utf8mb4_general_ci        | 32.0 KiB         | -          |
| <input type="checkbox"/> invoice    |            | 3         | InnoDB        | utf8mb4_general_ci        | 32.0 KiB         | -          |
| <input type="checkbox"/> service    |            | 4         | InnoDB        | utf8mb4_general_ci        | 16.0 KiB         | -          |
| <input type="checkbox"/> staff      |            | 3         | InnoDB        | utf8mb4_general_ci        | 32.0 KiB         | -          |
| <b>6 tables</b>                     | <b>Sum</b> | <b>22</b> | <b>InnoDB</b> | <b>utf8mb4_general_ci</b> | <b>208.0 KiB</b> | <b>0 B</b> |

Figure 4: Row counts for all six tables after loading the sample data

## 6. SQL Queries and Results

Seven queries were run, in the order shown. Some depend on an earlier one. For example, query 2 assigns the staff member created by query 1. Each query below has its purpose, the SQL code, an explanation and the screenshot of the result.

### 6.1 Query 1: Register a New Staff Member (INSERT)

The first query adds a new staff member, Mariam Youssef, who joins as a Cleaner:

```
INSERT INTO Staff (full_name, phone, role, hire_date)
VALUES ('Mariam Youssef', '05099990004', 'Cleaner', '2026-09-20');
```

staff\_id is left out on purpose, since that column fills itself in. Staff 1 to 3 were already in the table, so Mariam naturally becomes staff number 4. phpMyAdmin confirmed this with "1 row inserted" and "Inserted row id: 4", shown below.

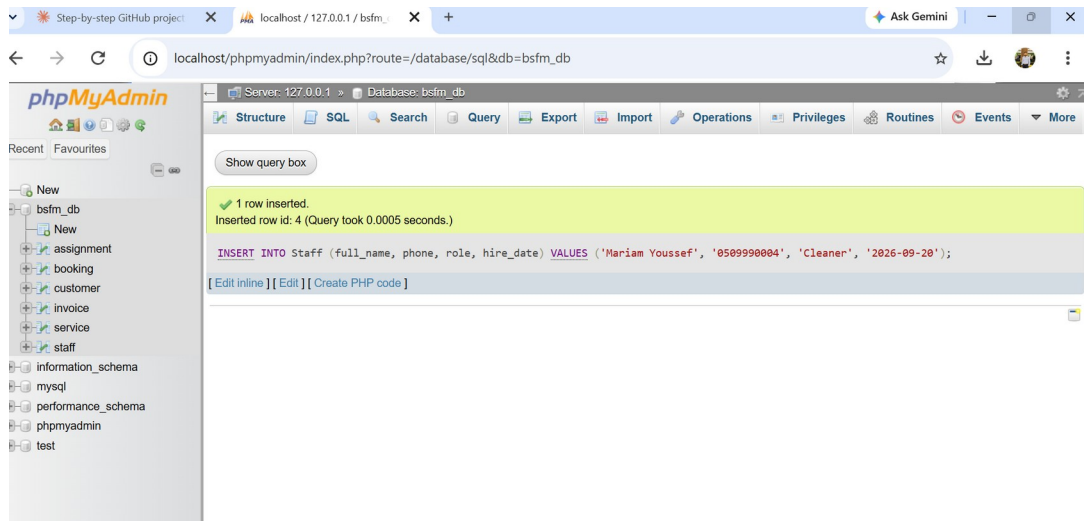


Figure 5: Result of the INSERT into Staff

## 6.2 Query 2: Assign the New Staff Member to a Booking (INSERT)

Now that Mariam exists in the Staff table, the next step is to put her on a job. This query assigns her (staff\_id 4) to booking 2, which is Fatima Al Suwaidi's AC Duct Cleaning appointment:

```
INSERT INTO Assignment (booking_id, staff_id, assigned_date, role_in_job)
VALUES (2, 4, '2026-09-26', 'Assistant');
```

This only works because of the foreign keys on Assignment — MySQL checks that booking 2 and staff 4 both actually exist before it lets the row in. Assignments 1 through 4 already existed from the sample data, so this became assignment 5, and "1 row inserted" appeared again.

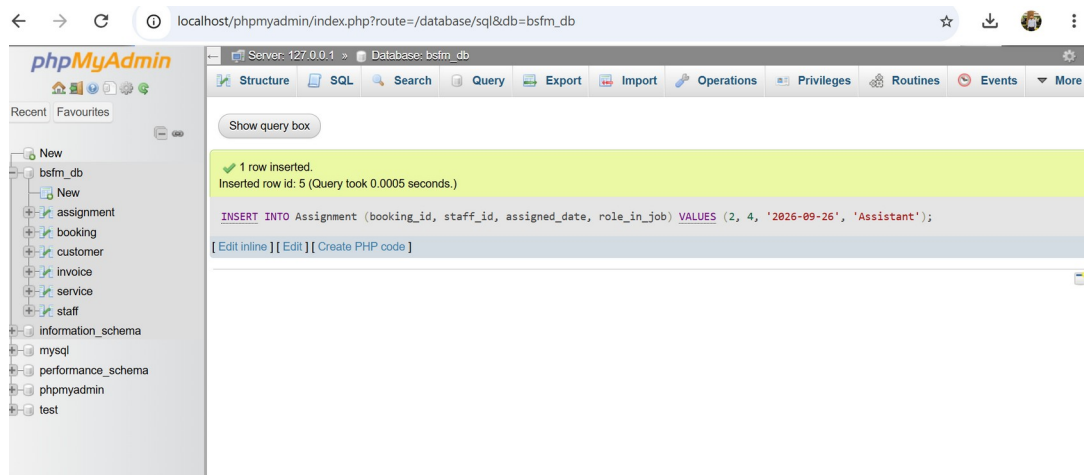


Figure 6: Result of the INSERT into Assignment

## 6.3 Query 3: Assignment History of One Staff Member (SELECT with JOIN)

This one answers a more useful question: what has a specific staff member actually worked on? Looking at Yusuf Rahman (staff\_id 1):

```
SELECT a.assignment_id, s.service_name, a.assigned_date, a.role_in_job
FROM Assignment a
JOIN Booking b ON a.booking_id = b.booking_id
```

```

JOIN Service s ON b.service_id = s.service_id
WHERE a.staff_id = 1
ORDER BY a.assigned_date DESC;

```

The Assignment table only knows a booking number, not what service that booking is for — so it takes two JOINS to get there: one to reach the booking, and a second from the booking to its service name. Filtering on `staff_id = 1` and sorting by date descending gives Yusuf's two jobs with the most recent one first: assignment 3 (Sofa and Carpet Cleaning, 27 Sep, as Assistant) and assignment 1 (Residential Deep Cleaning, 25 Sep, as Lead).

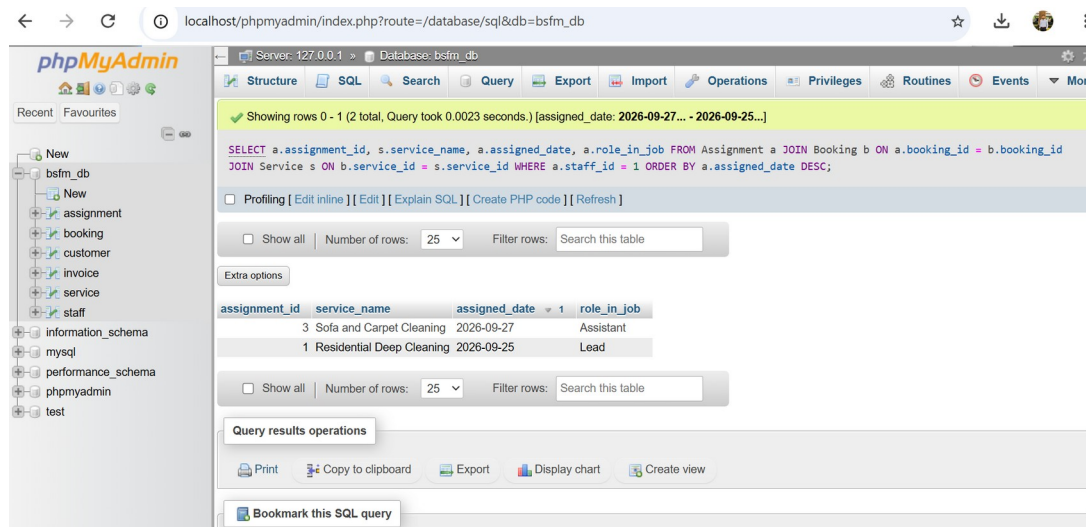


Figure 7: Result of the assignment history query

## 6.4 Query 4: Unpaid Invoices with Customer Name (SELECT with Two JOINS)

This query is meant for the finance side of things — which customers still owe money, and how much:

```

SELECT c.full_name, i.invoice_id, i.amount, i.issue_date
FROM Invoice i
JOIN Booking b ON i.booking_id = b.booking_id
JOIN Customer c ON b.customer_id = c.customer_id
WHERE i.payment_status = 'Unpaid'
ORDER BY i.issue_date;

```

Three tables are involved here: Invoice for the amount and status, Booking as the bridge, and Customer for the name that actually needs to be shown. The WHERE clause keeps only rows where `payment_status` is Unpaid. In the current data that is a single invoice — Ahmed Al Mazrouei, invoice 3, for 900.00, issued on 28 September (booking 4, the Office Cleaning Contract).

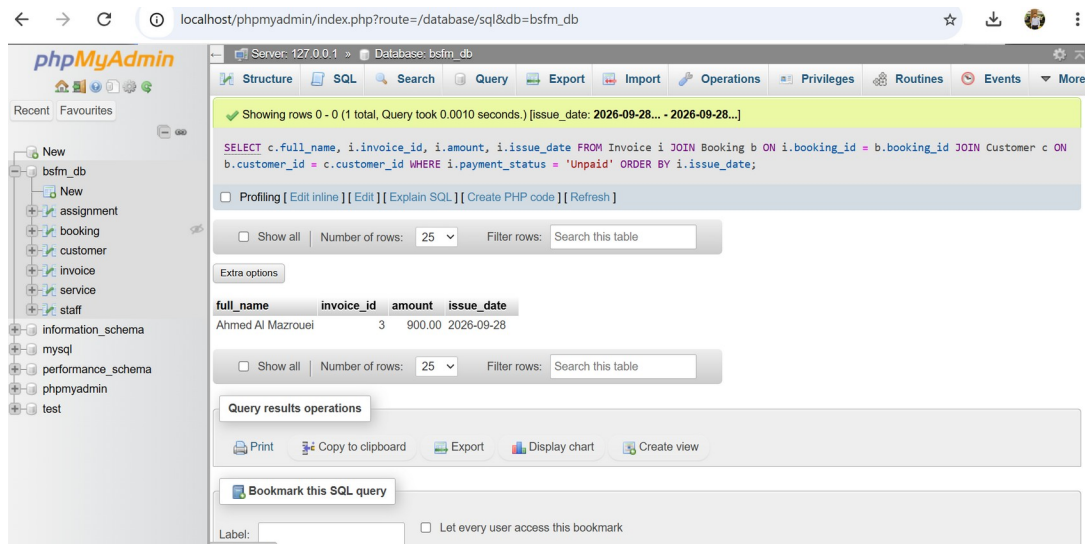


Figure 8: Result of the unpaid invoices query

## 6.5 Query 5: Mark an Invoice as Paid (UPDATE)

Once that invoice above gets settled, its status needs to change. UPDATE handles that:

```
UPDATE Invoice
SET payment_status = 'Paid'
WHERE invoice_id = 3;
```

The WHERE clause here is doing the important work — without it, every row in Invoice would flip to Paid, which is obviously not what was intended. Limiting it to `invoice_id = 3` keeps the change to exactly the one invoice from Query 4. phpMyAdmin reported "1 row affected", confirming only that single row changed.

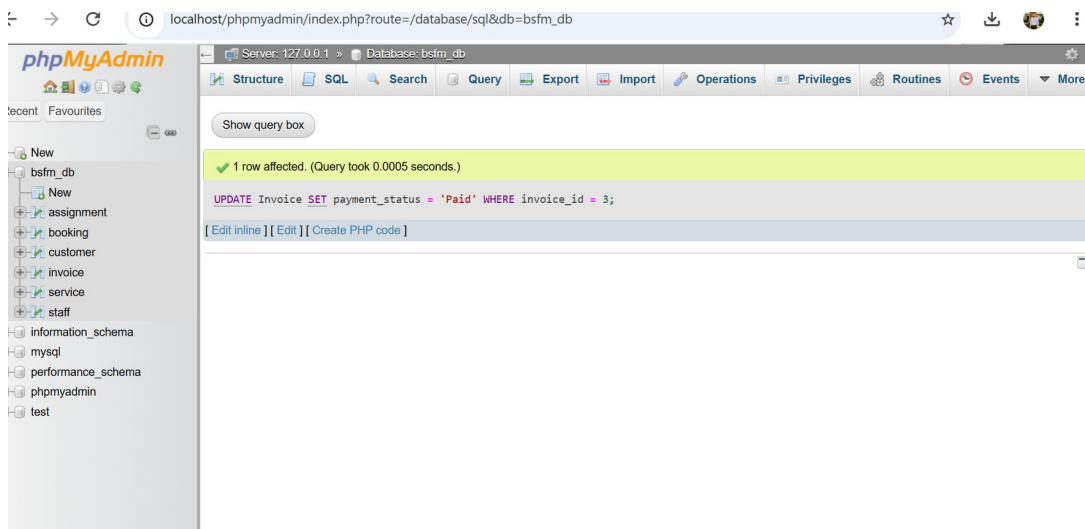


Figure 9: Result of the UPDATE on Invoice

## 6.6 Query 6: Remove an Incorrect Assignment (DELETE)

Not every entry stays correct forever — this query removes assignment 3, an Assistant-role entry that turned out to have been logged by mistake:

```
DELETE FROM Assignment
```

```
WHERE assignment_id = 3 AND role_in_job = 'Assistant';
```

Two conditions are chained together with AND, so the delete only goes through if both the ID and the role match. That second condition is really just a safety net — it makes sure the wrong row can't accidentally get deleted if the ID alone was mistyped. "1 row deleted" came back, meaning the operation removed exactly what it was supposed to.

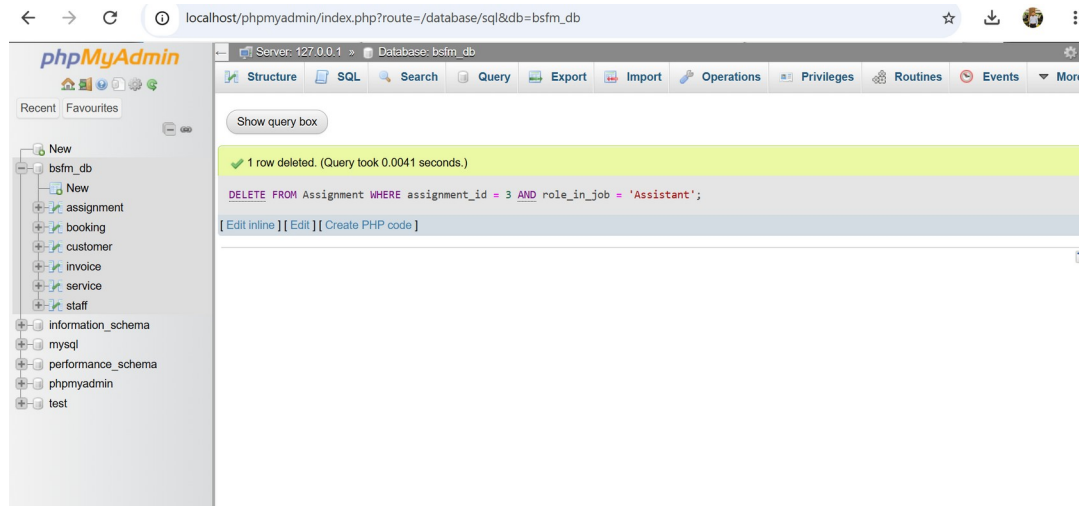


Figure 10: Result of the DELETE on Assignment

## 6.7 Query 7: Assignments per Staff Role (Aggregate)

The last query steps back and looks at the bigger picture — how much work is each role handling overall:

```
SELECT st.role, COUNT(a.assignment_id) AS total_assignments
FROM Assignment a
JOIN Staff st ON a.staff_id = st.staff_id
GROUP BY st.role;
```

The JOIN tags every assignment with the role of whoever is doing it, and GROUP BY st.role sorts those into one bucket per role. COUNT(a.assignment\_id) then just counts how many assignments landed in each bucket. After Query 6 removed assignment 3, the four that remain are 1 (Yusuf, Cleaner), 2 (Sara, Technician), 4 (Hassan, Supervisor) and 5 (Mariam, Cleaner) — which is why Cleaner comes out on top with two, and the other two roles have one each.

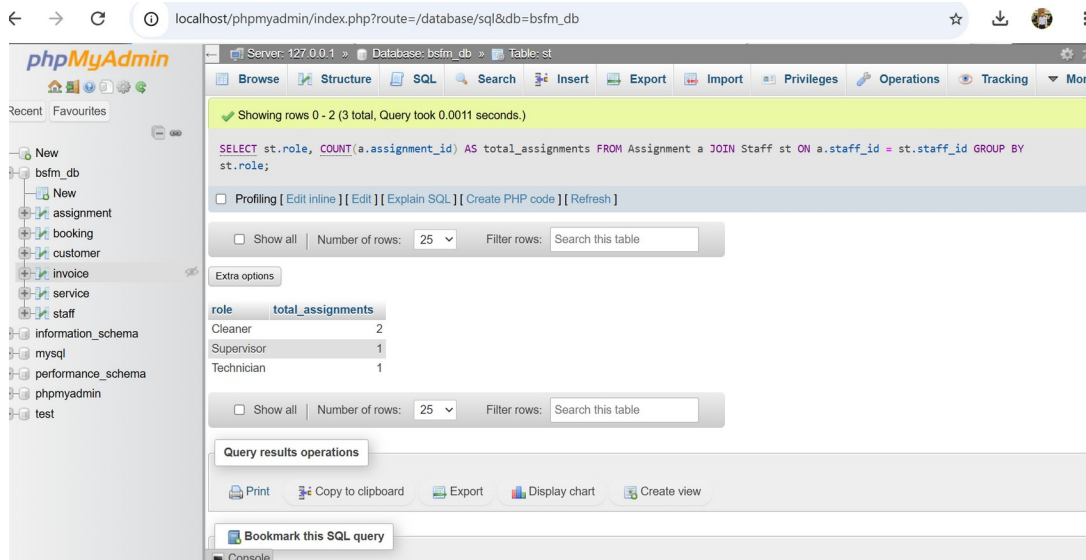


Figure 11: Result of the assignments per role query

## 6.8 Summary of Results

| No. | Operation                        | Tables used                  | Result                                         |
|-----|----------------------------------|------------------------------|------------------------------------------------|
| 1   | INSERT (create)                  | Staff                        | 1 row inserted (staff 4)                       |
| 2   | INSERT (create)                  | Assignment                   | 1 row inserted (assignment 5)                  |
| 3   | SELECT with two JOINS (read)     | Assignment, Booking, Service | 2 rows                                         |
| 4   | SELECT with two JOINS (read)     | Invoice, Booking, Customer   | 1 row                                          |
| 5   | UPDATE                           | Invoice                      | 1 row affected                                 |
| 6   | DELETE                           | Assignment                   | 1 row deleted                                  |
| 7   | SELECT with GROUP BY (aggregate) | Assignment, Staff            | 3 rows (Cleaner 2, Supervisor 1, Technician 1) |

## 7. Notes and Observations

- Order matters. Some statements use fixed ID numbers, such as staff 4 in query 2 and assignment 3 in query 6. They work on a fresh database when run in order. Running the sample data a second time would change the numbers and break them.
- Invoice is one-to-one with Booking. The UNIQUE constraint on Invoice.booking\_id means a booking can never be invoiced twice; a second INSERT with the same booking\_id would be rejected.
- Assignment is many-to-many in effect. A booking can have more than one staff member assigned (as seen with staff 1 and 4 both linked to different bookings), and a staff member can be assigned to more than one booking.
- Rules that protect the data. The unique phone number prevents duplicate staff records, the ENUM restricts payment\_status to three valid values, DECIMAL stores invoice amounts exactly, and the foreign keys prevent assignments or invoices for bookings or staff that do not exist.
- Shared tables. Customer, Service and Booking were designed separately but are used here through JOINS, since Assignment and Invoice both depend on Booking.

## 8. GitHub Repository

All scripts and screenshots are stored in the shared team repository:

<https://github.com/shakhawat1194/bsfm-shakhawat-database>

| Path                           | Contents                                         |
|--------------------------------|--------------------------------------------------|
| sql/04_person2_tables.sql      | Creates the Staff, Assignment and Invoice tables |
| sql/05_person2_sample_data.sql | Inserts the Person 2 sample data                 |
| sql/06_person2_queries.sql     | The seven queries described in this report       |
| screenshots/p2-*.png           | The eleven screenshots used in this report       |
| README.md                      | Project overview and results                     |

## 9. Conclusion

The Staff, Assignment and Invoice tables were designed, created and filled with sample data for BSFM's booking system, extending the Customer, Service and Booking tables built earlier. Primary keys, foreign keys, a unique phone number, a unique invoice-per-booking rule and a restricted payment status list keep the data consistent.

Seven queries were run and checked against their expected results. They cover all four basic operations (create, read, update and delete), two JOIN reports that give a staff member's assignment history and the list of unpaid invoices with customer names, and one aggregate query that counts assignments per staff role. Every result is documented with a screenshot, and all scripts are available in the GitHub repository.